LOCKWISE

Fase I — sabatina e vídeo pitch

ExpoTech 2026.2 · Categoria INTERFACE
UniFECAF · Engenharia da Computação

Avaliação individual: 09 a 13 de novembro de 2026
Evento: 28 de novembro de 2026

Integrante	RA
Rodrigo Gonçalves da Silva	98550
Carlos Henrique Barreto	113749
Fabrício Conceição Silva	114204

Este documento reúne o banco de perguntas da sabatina e o roteiro do vídeo pitch. Gerado a partir de `docs/sabatina.md` e `docs/video-pitch.md` do repositório.

Parte I — Sabatina

Fase I. Material de ensaio para a avaliação individual de 09 a 13/11/2026.

O regulamento dá 15% à sabatina, mas o peso real está na observação ao lado da tabela: **resposta vaga elimina a elegibilidade do grupo inteiro**. Não se perde 15 pontos — caem os três juntos. A nota é individual e a falta de um integrante atribui zero a ele.

Por isso este documento não divide o projeto em três pedaços. Cada um de nós precisa saber responder sobre **qualquer parte**, inclusive a que não construiu. A divisão que existe é só de ordem de estudo.

Como ensaiar

Uma pessoa pergunta, as outras duas respondem — alternando quem responde primeiro, para ninguém se acostumar a completar a frase do outro. Três rodadas, de dificuldade crescente:

1. **Parte 1** (o projeto inteiro). Todos respondem tudo, de cabeça.
2. **Partes 2 a 6** (técnica por eixo). Sorteiem quem responde cada uma.
3. **Parte 7** (as perguntas ruins). Essas são as que decidem.

Regra do ensaio: quem responder "acho que", "mais ou menos" ou "foi o Rodrigo que fez essa parte" repete a pergunta no fim da rodada. Na avaliação real essas três frases são exatamente o que o regulamento chama de resposta vaga.

Parte 1 — O projeto (todos precisam saber de cor)

O que é o LOCKWISE?

> Um sistema de controle de acesso em que uma fechadura digital feita de portas lógicas decide quem entra, e o mesmo evento de acesso atravessa um gateway, uma API na nuvem e um modelo de otimização, voltando como escala de vigilância.

Se sobrar tempo, a frase que amarra tudo: **é um dado só, atravessando cinco camadas.**

Por que vocês criaram esse projeto?

> A categoria INTERFACE pede circuito digital, física, arquitetura de software, nuvem e otimização. A saída fácil seria cinco trabalhos separados entregues na mesma pasta. Escolhemos um tema em que as cinco partes precisassem umas das outras: o acesso que o circuito decide é o mesmo acesso que a nuvem guarda e o mesmo que o modelo consome.

Complemento, se perguntarem por que controle de acesso e não outro tema:

> Dos temas sugeridos, era o único **inteiramente digital**. Os outros partem de grandezas analógicas, e o Logisim não simula analógico. Começar por um tema que a ferramenta não comporta seria gastar o semestre contornando a escolha.

Por que ele é útil?

> Uma fechadura decide quem entra, mas quem administra um prédio também precisa saber quantas pessoas escalar em cada turno. No LOCKWISE o histórico que a porta gera é a demanda do modelo de escala, então o registro de acesso vira uma decisão de pessoal.

O número concreto, para não ficar abstrato:

> No cenário do projeto, com histórico sintético, a escala ótima usa **9 vigilantes contra 12** da escala ingênua: R$ 748,00 por dia a menos, 26,6%.

Como ele se diferencia?

Aqui é onde se perde ponto por exagero. A resposta honesta tem duas partes.

> **O que não é diferencial:** fechadura com senha, registro em nuvem e painel de histórico existem no mercado, e bem feitos.
>
> **O que é nosso diferencial:** o laço fechado até o dimensionamento de pessoal, e as garantias serem verificáveis em vez de afirmadas. A equivalência entre o circuito e o software é provada contra o arquivo do circuito em 2048 casos, não prometida. A arquitetura em camadas é imposta por um teste que lê a árvore sintática, não descrita num diagrama. O ótimo do modelo é conferido por força bruta, não aceito do solver. E o deploy é validado pela SHA do commit que chegou ao ar, não por um "status: ok".

Se insistirem em "mas isso vende?":

> É um projeto acadêmico, e não medimos mercado. O que afirmamos é técnico: são quatro garantias conferidas de forma automática, por teste ou pelo pipeline.

Qual foi a parte mais difícil?

Pergunta de aquecimento, mas revela quem construiu e quem assistiu. Cada um precisa de uma resposta própria e verdadeira. Exemplos que servem:

> Fazer o bloqueio acontecer na terceira tentativa e não na quarta. O flip-flop só assume o valor novo depois da borda, então o sinal de limiar precisa ser calculado antes dela.

> Descobrir que o deploy estava verde com a versão antiga no ar. O teste checava `"status":"ok"`, que é verdade em qualquer versão.

Quem fez o quê?

Esta resposta o grupo escreve junto, com a divisão real do trabalho. Cada um diz o que de fato fez e completa:

> Mas qualquer um de nós responde sobre qualquer parte, e é por isso que ensaiamos trocando.

Nunca responder "essa parte não foi comigo". É a frase que derruba o grupo.

Parte 2 — Circuito digital

Como o circuito decide que a senha está certa?

> Comparando a senha nos pinos S3–S0 com 1011, que é a senha gravada. O comparador tem dois estágios: `P1 = S3 · ¬S2` confere a metade alta e `P2 = S1 · S0` a metade baixa. `IGUAL = P1 · P2`.

Por que dois estágios em vez de um AND de quatro entradas?

> Para reduzir o fan-in e evitar um AND de 4 entradas. Dois ANDs de 2 entradas e um terceiro que os junta fazem a mesma comparação.

Quais são os estados e como são codificados?

Estado	Q1 Q0	O que significa
AGUARDANDO	00	repouso
VERIFICANDO	01	a senha está sendo comparada
LIBERADO	10	trava acionada
BLOQUEADO	11	travado, só RESET sai

> São dois flip-flops D. A decodificação é direta: `AG = ¬Q1·¬Q0`, `VER = ¬Q1·Q0`, `LIB = Q1·¬Q0`, `BLQ = Q1·Q0`.

Por que `E = C1 · (C0 + ERRO)` e não simplesmente `C1 · C0`?

Esta é uma das duas perguntas que mais esperamos.

> Porque o contador só atinge `11` **depois** da borda de clock. Se o bloqueio dependesse apenas do valor já armazenado, no instante do terceiro erro a máquina veria `E = 0` e só bloquearia no quarto erro. A expressão antecipa o limiar: com `ERRO = 1`, `E` vale `C1`, ou seja, "já houve duas tentativas".

Por que o contador satura em vez de dar a volta?

A outra pergunta esperada.

> Porque `EN = ERRO · ¬(C1·C0)`. Com o contador cheio, ele desabilita a própria contagem. Sem essa realimentação, um erro a mais levaria `11` de volta a `00` e o sistema liberaria novas tentativas sozinho — exatamente o que o bloqueio existe para impedir.

Por que construíram o contador com flip-flops e não com o componente Counter pronto?

> Porque o componente pronto é uma caixa que não dá para explicar numa avaliação individual. Com flip-flops D, somador de 1 e porta de habilitação, cada porta tem um motivo e uma equação.

O que acontece se eu digitar a senha certa com o sistema bloqueado?

> Nada. Em BLOQUEADO, a única transição de saída depende de RESET. `IGUAL` continua sendo calculado, mas não participa do próximo estado nesse ramo.

Quantas peças tem o circuito?

> 37: 33 portas lógicas e 4 flip-flops D. Nenhum componente pronto de contagem ou comparação.

Parte 3 — Física e eletrônica

De onde vêm os 18 200 mJ de cada acesso?

> De 3,53 watts durante 5,16 segundos. A potência soma a bobina, 12 V × 292,5 mA = 3,51 W, e a base do transistor, 5 V × 4,3 mA = 0,02 W. O tempo é o do NE555, T = RC·ln 3. O produto dá cerca de 18,2 J, e cada fator tem cálculo próprio no memorial de física.

Cuidado com a forma curta: 12 V × 292,5 mA × 5,16 s dá 18 112 mJ, porque deixa a base de fora.

Por que não ligar a trava direto na saída da porta lógica?

> Porque a bobina exige 300 mA e a porta fornece 25 mA no máximo absoluto, 12 vezes menos. E porque a trava é de 12 V e a lógica de 5 V: mesmo sem queimar, a força cairia a 17% e a trava não abriria. Por isso há um estágio de potência com um BC337 em saturação.

Para que serve o diodo?

> Quando o transistor corta, a bobina gera uma tensão reversa para manter a corrente. Sem o diodo, essa tensão chega a centenas de volts e destrói o transistor, que suporta 45 V. O 1N4007, em antiparalelo com a bobina, dá caminho à corrente e limita a tensão a 12,8 V.

Como escolheram o resistor de base?

> Não pelo h_FE da folha de dados, que vale para a região ativa. Impusemos um ganho forçado três vezes menor que o h_FE mínimo, para garantir saturação, e calculamos R_B no pior caso de tensão de saída da porta. O resultado de 1 kΩ dá de 3,7 a 4,3 mA, dentro do que a porta 74HC fornece com nível garantido.

O circuito está montado?

Responder isto sem rodeio. Ver também a Parte 7.

> Não. Está integralmente dimensionado, com cálculo térmico e de saturação, mas não foi montado em bancada. Está declarado no memorial e no relatório. O roteiro da categoria aceita circuito simulado.

Parte 4 — Gateway e a prova de equivalência

O Logisim conversa com a internet?

> Não, e nunca conversou. Ele não expõe o estado da simulação para fora e não há caminho suportado para isso.

Então como o evento sai do circuito?

> O gateway é a mesma máquina de estados reescrita em Python, porta a porta. Quem opera o circuito repete o mesmo gesto no gateway, e é o gateway que faz o POST.

Isso não é só confiar que os dois são iguais?

A pergunta mais perigosa deste eixo. A resposta é o diferencial do projeto.

> Não. Há um teste que lê o arquivo `.circ`, reconstrói a netlist e compara com o Python em **todas** as combinações possíveis: 128 de pinos × 16 de registradores = 2048 casos, com zero divergências. A equivalência é provada contra o arquivo do circuito, não afirmada.

Por que não usar um microcontrolador lendo os pinos?

> A equipe decidiu não seguir com a simulação em microcontrolador. O circuito não está montado, então não há pino físico para ler, e a ponte é o gêmeo digital, com a equivalência provada por teste.

Parte 5 — Nuvem, arquitetura e processo

Onde a API roda e por quê?

> FastAPI no Render, com Postgres gerenciado na Neon. Render porque faz deploy direto do GitHub, sem cartão no plano gratuito. AWS custaria dias de configuração de permissões antes da primeira linha útil. O banco foi para a Neon porque o Render permite um único Postgres gratuito por conta, e a conta já tinha um, de outro projeto.

Como a API sabe o estado atual da fechadura?

> Reconstruindo a partir dos eventos. É a mesma máquina de estados pela terceira vez — no circuito ela decide o presente, no gateway ela acompanha, e na API ela projeta o passado. Na nuvem não existe VERIFICANDO, porque ela só recebe o resultado da verificação.

Que padrões de projeto vocês usaram?

> Três padrões GoF, cada um com um trabalho real. **State** (`dominio/estados.py`): a nuvem reconstrói o estado da fechadura a partir dos eventos. **Strategy** (`dominio/politicas.py`): a política de quando avisar alguém é trocada por variável de ambiente, sem mexer no serviço. **Observer** (`dominio/notificacao.py`): quem cria o alerta não conhece os interessados, e um observador que falha não derruba o registro do acesso.

Como vocês garantem que a arquitetura em camadas foi respeitada?

> Por um teste que lê a árvore sintática do código e falha se uma camada importar outra que não deveria. A arquitetura é imposta, não descrita.

Já tiveram algum problema sério de processo?

Pergunta boa de responder bem — mostra maturidade.

> Sim, e está registrado numa ADR. Uma dependência sem teto resolveu para uma versão nova e incompatível, o CI quebrou, e como o CI barra o deploy, um commit ficou duas semanas sem chegar à produção sem ninguém perceber. A causa não foi a dependência: foi rodar o teste local, dizer "verde" e não olhar o CI. Hoje fixamos a faixa da dependência e conferimos o pipeline depois de cada push.

Como vocês sabem que o que está no ar é o commit certo?

> O `/health` devolve a SHA do commit, e o deploy compara. Antes disso o teste checava `"status":"ok"`, que é verdade em qualquer versão — foi assim que ficamos com a versão antiga no ar e o pipeline verde.

Por que tantas ADRs?

> Porque numa sabatina a pergunta mais provável é "por que vocês fizeram assim?". São 32 decisões registradas com o que estava em jogo, o que descartamos e o que a escolha nos custa.

Parte 6 — Otimização

O que exatamente o modelo decide?

> Quantos vigilantes escalar em cada um dos seis turnos de 8 horas, ao menor custo, sem deixar nenhuma hora do dia descoberta. É programação linear inteira, resolvida com PuLP e CBC.

De onde vem a demanda?

> Do histórico de acessos que o próprio circuito gerou, agregado por hora e dividido pelos dias do período. Não é demanda inventada.

Por que turnos de 8 horas começando a cada 4?

> Porque assim **cada hora é coberta por dois turnos**, e escalar alguém no turno 12–20 ajuda a tarde e o começo da noite. É o que torna a decisão um problema combinatório de verdade, em vez de uma divisão. E 8 horas é o que um vigilante trabalha.

Como se converte acesso em vigilante?

> Pela capacidade K: estimando quinze minutos de atenção por acesso — conferir, registrar, acompanhar —, um vigilante dá conta de quatro por hora. K = 4.

Por que o modelo deixa turnos vazios e sobra gente de madrugada?

> Porque a restrição proíbe **hora descoberta**, não turno vazio. E entre 4h e 6h sobram vigilantes porque com blocos de 8 horas não existe contratar alguém para duas horas. É o custo da granularidade, e ele aparece explícito na tabela de cobertura.

Como vocês sabem que o solver achou o ótimo?

> Um teste confere por força bruta, enumerando as combinações possíveis no cenário de teste. O resultado do solver não é aceito por confiança.

Parte 7 — As perguntas ruins

Estas são as que decidem a sabatina. Em todas, a estratégia é a mesma: **admitir primeiro, explicar depois**. Avaliador fareja defensiva.

"Isso não é um protótipo fictício? É tudo simulação."

> O circuito é simulado, e isso está declarado. Mas a cadeia é real de ponta a ponta: a API está no ar com banco gerenciado, o pipeline barra o deploy, e o evento que o senhor vai ver nascer no circuito aparece no banco na nuvem agora, na sua frente. É o que vamos demonstrar.

Este é o risco nº 1 do projeto. A defesa não é verbal — é a demo encadeada ao vivo. Se virar "Logisim numa tela e print da API na outra", a resposta não salva.

"A economia de R$ 22.440 por mês é crível?"

A pergunta mais perigosa do projeto. Admitir o limite antes de defender o número.

> Ela vale para o cenário do modelo e não é uma medição. A escala otimizada custa R$ 2.068 por dia, com 9 vigilantes. A ingênua custa R$ 2.816, com 12. A diferença é R$ 748 por dia, 26,6%, ou R$ 22.440 em 30 dias.
>
> O ponto fraco é o dimensionamento. K = 4, quinze minutos por acesso, é uma estimativa nossa, e ela exige 4 vigilantes no pico para um condomínio de cerca de 50 unidades, o que é alto. Com K = 6, a escala cai para 6 vigilantes e R$ 1.386 por dia.
>
> O custo de R$ 220 por turno também é uma ordem de grandeza. A referência pública do CadTerc equivale a cerca de R$ 376 por 8 horas. Trocar esse valor muda os reais, mas não muda a escala nem o percentual.

Cuidado com os R$ 84.480: esse valor é o custo mensal da escala ingênua (R$ 2.816 × 30), não a economia.

"Por que o circuito não está montado?"

> O roteiro aceita circuito simulado, e isso está declarado no memorial e no relatório. O estágio de potência está dimensionado, com cálculo térmico e de saturação. Montá-lo custaria cerca de R$ 30 em componentes e é o caminho mais direto para fechar essa lacuna.

"Vocês usaram IA para fazer isso?"

Responder com naturalidade; a pergunta é sobre domínio, não sobre a ferramenta.

> Usamos, como ferramenta. O que garante que entendemos o que está aqui não é a origem do código — é que cada decisão está registrada em ADR com a alternativa descartada, e que as garantias são verificadas por teste. Pergunte qualquer porta do circuito e eu explico o papel dela.

E então é preciso conseguir fazer isso. É o motivo da Parte 2 existir.

"Me explique essa porta aqui."

Vale apontar qualquer peça do circuito. A página da placa em 3D mostra a equação e a coordenada de cada uma, e serve de material de estudo.

> (treinar com a placa aberta, apontando portas ao acaso)

"O histórico é sintético. Então os dados são inventados?"

> Os eventos nascem da mesma máquina de estados do gateway e entram pela mesma API. O `tentativa`, o `energia_mj` e a ordem dos estados estão corretos porque quem os produz é o circuito replicado. O que difere da operação real é só o relógio.

"Se eu tirar a internet, o que acontece?"

> A fechadura continua decidindo, porque quem decide é o circuito, com o comparador e a máquina de estados, não a nuvem. O gateway tem fila offline e reenvia quando a API volta. O que se perde é o registro em tempo real, não o controle de acesso.

Antes da avaliação

- ■ Ensaiar as três rodadas pelo menos duas vezes, trocando quem pergunta
- ■ Cada um escolher sua resposta verdadeira para "qual foi a parte mais difícil"
- ■ Rodar a demo encadeada inteira, cronometrada, sem pular passos
- ■ Acordar a API antes (o plano gratuito hiberna)
- ■ Conferir que o CI está verde no dia
- ■ Abrir a placa em 3D e treinar apontando portas ao acaso

Parte II — Roteiro do vídeo pitch

Três minutos, exigidos pelo regulamento geral da ExpoTech (não pelo roteiro da INTERFACE, e é por isso que costuma ser esquecido).

Três minutos é pouco. A escolha aqui é **mostrar o sistema funcionando** em vez de narrar o que ele faz: quem assiste precisa ver o dado nascer no circuito e chegar na nuvem. Slides explicativos consomem tempo e não provam nada.

Formato

Captura de tela com narração em voz. Ninguém aparece. Sem webcam, sem câmera, sem foto da equipe — a tela é a única imagem do vídeo inteiro. Isso combina com a escolha acima: os três minutos ficam todos para o sistema.

Narram o Carlos e o Fabrício. O Rodrigo não participa da gravação. Isso não tem efeito na nota individual: o regulamento avalia domínio individual na sabatina, em sala, não no vídeo. Mas tem um efeito prático — como nenhum rosto aparece, **cada um se identifica pela voz na primeira fala**, senão quem assiste não sabe quem é quem.

A equipe inteira, com os três nomes e RAs, aparece escrita na tela do fecho.

Estrutura

Tempo	Bloco	Narra	O que aparece na tela
0:00–0:20	O problema	Carlos	Fala, sobre a tela do circuito parado
0:20–0:40	O circuito	Carlos	Logisim, senha correta, trava abre
0:40–1:10	O bloqueio	Carlos	Três erros, contador subindo, BLOQUEADO
1:10–1:50	A ponte	Fabrício	Gateway ao lado, POSTs saindo, 201
1:50–2:20	A nuvem	Fabrício	Navegador: `/fechadura`, `/acessos`, banco
2:20–2:50	A otimização	Fabrício	Relatório com os 26,6%
2:50–3:00	Fecho	Carlos	Equipe e o repositório

A divisão segue o assunto, não o cronômetro: o **Carlos** fica com o hardware — o circuito, o bloqueio e a física que os sustenta — e o **Fabrício** com o software e a matemática — a ponte, a nuvem e a otimização. Dá 80 segundos de fala para um e 100 para o outro.

Essa divisão vale **só para o vídeo**. Na sabatina cada um responde sobre qualquer parte; ver `docs/sabatina.md`.

Texto

0:00–0:20 · O problema — Carlos

> Aqui é o Carlos, do projeto LOCKWISE.
>
> Uma fechadura eletrônica precisa decidir sozinha quem entra. Mas a decisão dela, sozinha, não vale muito: quem administra um prédio quer saber **quando** as pessoas entram, **quantas** tentativas falharam, e **quantos vigilantes** precisa ter em cada turno.
>
> O LOCKWISE liga essas duas pontas. Um circuito digital decide o acesso, e o mesmo evento atravessa até virar uma escala de trabalho.

Tela: o circuito no Logisim, parado, com AGUARDANDO aceso.

0:20–0:40 · O circuito

> O circuito compara uma senha de quatro bits e governa uma máquina de estados de quatro estados. Tudo em portas lógicas e flip-flops — nenhum componente pronto.

Tela: digitar `1011`, SENHA_OK acende. CONFIRMA, clock → VERIFICANDO. Clock → TRAVA_ABERTA.

> Senha certa: a trava abre.

0:40–1:10 · O bloqueio

> Agora o contrário.

Tela: três tentativas erradas, mostrando o contador `01` → `10` → `11`.

> Um erro, dois erros. No terceiro, o contador satura e o sistema bloqueia. E repare: **o bloqueio acontece na terceira tentativa, não na quarta**. O sinal de limiar é calculado antes da borda de clock, porque o flip-flop só assume o valor novo depois dela.

Tela: tentar `1011` com o sistema bloqueado — não adianta. RESET libera.

> Nem a senha certa destrava. Só a intervenção administrativa.

1:10–1:50 · A ponte — Fabrício

> Aqui é o Fabrício.
>
> O Logisim não fala com a internet. Então o gateway é a mesma máquina de estados escrita em Python — e a equivalência não é promessa, é teste.

Tela: rodar `pytest tests/test_netlist.py`, mostrar verde.

> Esse teste lê o arquivo do circuito, reconstrói a netlist e compara com o Python em **todas** as combinações possíveis de estado e entrada.

Tela: duas janelas lado a lado — Logisim e gateway. Repetir a senha correta nos dois.

> Mesmo gesto dos dois lados. E aqui o evento sai para a nuvem.

Tela: `POST /acessos → enviado HTTP 201`.

1:50–2:20 · A nuvem

Tela: navegador em `lockwise-api.onrender.com/fechadura`.

> A API está no Render, com Postgres gerenciado na Neon. Ela reconstrói o estado da fechadura a partir dos eventos — é a mesma máquina de estados pela terceira vez, agora projetando o passado.

Tela: `/acessos` mostrando o registro recém-criado, com o `energia_mj`.

> E cada liberação carrega dezoito mil e duzentos milijoules. Esse número não é enfeite: são três vírgula cinquenta e três watts, da bobina e da base do transistor, durante cinco vírgula dezesseis segundos — cada fator calculado no memorial de física.

2:20–2:50 · A otimização

Tela: rodar o `resolver`, deixar o relatório aparecer.

> O histórico de acessos vira a demanda de um modelo de programação linear inteira. Seis turnos de oito horas que se sobrepõem, e a pergunta: quantos vigilantes em cada um, ao menor custo, sem deixar hora descoberta?

Tela: destacar o bloco da economia.

> Nove vigilantes contra doze da escala ingênua. Vinte e seis por cento mais barato — vinte e dois mil reais por mês.

2:50–3:00 · Fecho — Carlos

Tela: página do repositório no GitHub, com os três nomes e RAs sobrepostos — Rodrigo Gonçalves da Silva (98550), Carlos Henrique Barreto (113749), Fabrício Conceição Silva (114204).

> Circuito, física, arquitetura, nuvem e otimização. Um evento atravessando as cinco.
>
> LOCKWISE. Código e decisões, abertos no GitHub.

Antes de gravar

- ■ Acordar a API (`/health` no navegador) — o plano gratuito hiberna
- ■ Semear o histórico, se o banco estiver vazio
- ■ Logisim com o circuito aberto e a simulação reiniciada
- ■ Gateway já apontando para produção (`.\demo.ps1`) e **acordado**
- ■ Fonte do terminal aumentada — texto pequeno não se lê em vídeo
- ■ Ensaiar uma vez inteiro, cronometrando

Erros que custam caro

Falar sobre em vez de mostrar. Cada segundo de slide é um segundo a menos de sistema funcionando.

Estourar o tempo. Três minutos é limite, não sugestão. Se faltar tempo, corte o bloco do bloqueio (0:40–1:10) — é o mais longo e o menos essencial.

Gravar a tela pequena. O que não se lê no celular não existe.

Pressa no 1:50–2:20. É o trecho que prova que o sistema é real. Se algo tiver que respirar, é esse.

Divisão de falas

Blocos	Quem narra	Tempo
O problema · O circuito · O bloqueio · Fecho	**Carlos**	80 s
A ponte · A nuvem · A otimização	**Fabrício**	100 s

O Rodrigo não narra. Os três nomes aparecem na tela do fecho.

Gravar sem aparecer

Como ninguém aparece, tudo depende do que se lê na tela e do que se ouve.

Captura. Gravar a tela em 1920×1080, 30 fps. No Windows serve a Game Bar (`Win+G`) ou o OBS, que é melhor porque permite marcar a região e trocar de cena sem cortar. **Desligar qualquer sobreposição de webcam** — é o erro mais fácil de cometer e o único que invalida a escolha de formato.

Antes de gravar a tela: fechar notificações (modo foco), limpar a barra de tarefas, sair de contas pessoais visíveis no navegador, e aumentar a fonte do terminal e do editor. O que não se lê no celular não existe.

Voz. Gravar numa sala sem eco, com o microfone perto. Se as duas falas forem gravadas em lugares diferentes, deixar o volume parecido — corte com mudança brusca de som chama mais atenção que corte de imagem.

Montagem. Cada um grava seus blocos separado e junta depois. A troca de narrador cai sempre numa troca de tela (0:00→1:10 Carlos, 1:10→2:50 Fabrício, 2:50 Carlos), então a emenda fica escondida.

Nomear o arquivo: `lockwise-pitch.mp4`, até 3:00 cravados.

Anexo — Endereços do projeto

O quê	Endereço
Repositório no GitHub	`github.com/rodrigo-silvaXD/lockwise`
Simulador do circuito	`rodrigo-silvaxd.github.io/lockwise/`
Placa de portas em 3D	`rodrigo-silvaxd.github.io/lockwise/circuito-3d.html`
API — estado da fechadura	`lockwise-api.onrender.com/fechadura`
API — histórico de acessos	`lockwise-api.onrender.com/acessos`
API — saúde e SHA do commit	`lockwise-api.onrender.com/health`
API — documentação interativa	`lockwise-api.onrender.com/docs`
Pipelines (CI/CD)	`github.com/rodrigo-silvaXD/lockwise/actions`
Índice das ADRs	`github.com/rodrigo-silvaXD/lockwise/blob/main/docs/adr/README.md`

A API hiberna no plano gratuito: a primeira chamada pode levar cerca de um minuto para responder. Abrir `/health` antes de qualquer demonstração.